TravelCorp × Descope: Demo Guide

What the demo shows, the Descope flows behind it, and how to run it in your own project.

Overview

The TravelCorp demo is a working travel-booking portal built on Descope. It shows one customer identity reused across every booking, passwordless sign-in, sign-up directly from a booking form, households managed with Descope Family Accounts, and secure tools for a customer service team.

It is a reference implementation rather than production code. Every call to Descope can be inspected in the source, and the whole app runs against your own Descope project.

Stack: React web app (`@descope/react-sdk`) and a Node/Express API (`@descope/node-sdk`). The sign-in experience is built entirely in Descope Flows and embedded in the app, so you can change it in the Descope console without redeploying.

What the demo does

Page	What the customer or agent sees	Descope capabilities
Book a trip (`/book`)	A guest books without an account. "Continue to book" embeds the sign-up-or-in flow on the same page and carries the email and name across from the form. Apple Pay customers see "Sign in with Apple" first. After sign-in, the guest returns to the same booking and can choose family members under "Who's coming?".	Embedded flows, flow input (`form`), Family Accounts
Sign in (`/login`)	Passkeys where the device supports them; Apple and Google; an email code or link; a password with your policy; an optional authenticator app. Includes an A/B-tested welcome screen, a bot trap, and an English/French switch.	Passkeys, OAuth, OTP, Magic Link, Password, TOTP, A/B testing, bot trap, localization
My bookings (`/bookings`)	Every booking on one profile, shared across the household.	Session validation, Family Accounts claims
My family (`/family`)	Guardians sign in with their own logins. Dependents (children, pets) have no login. Guardians can add or remove dependents and "Act for" a dependent; the guardian is named in the issued token.	Family Accounts, dependent impersonation
My account (`/me`)	Self-service for name, email, phone, passkeys, password and authenticator app, plus a view of the validated session and its claims.	User Profile widget
Customer service (`/service`)	Agents with the Customer Service role find a customer and see their family, bookings and audit trail. They can view the account as the customer, with the agent named in the token.	RBAC, user search, user impersonation, audit search

Security model

- The API validates the Descope session on every request and authorizes from the token's claims: roles and permissions for customer service, and family membership for household actions.
- The web app reads the session only to decide what to show. Every authorization decision is enforced by the API.
- When an agent or a guardian acts for someone else, Descope issues a token for that user that also names the actor. Every action stays attributable to the real person.

Built-in flows

The sign-in experience is made of four Descope flows: one main journey and three subflows. They ship as JSON exports with the demo and can be imported into any project.

Flow	Role
`tp-sign-up-or-in`	Main journey: welcome screen, identifier-first routing, social login, onboarding

Flow	Role
`tp-email-verify`	Subflow: email verification by code or link
`tp-profile`	Subflow: collects the customer's name only when needed
`tp-mfa`	Subflow: optional authenticator-app (TOTP) enrollment and verification

`tp-sign-up-or-in` : the main journey

1. **Personalized welcome.** If the booking page reports an Apple Pay customer, the flow shows an Apple-first welcome screen. Otherwise a 50/50 A/B split chooses between an email-first and an Apple-first welcome screen, using Descope's built-in `abTestingKey`.
2. **Bot trap.** Both welcome screens include a hidden honeypot field that blocks automated sign-ups.
3. **Identifier-first routing.** After the customer enters an email address, the flow picks the strongest method available:
    - a **passkey**, if the customer has one *and* the device supports WebAuthn;
    - otherwise a **password**, if the customer has set one;
    - otherwise **email verification** (the `tp-email-verify` subflow).
4. **Social login.** Apple and Google sign-in go through Descope's OAuth support.
5. **Profile.** The `tp-profile` subflow makes sure there is a name on the account.
6. **Credential upgrade.** A customer with no passkey or password yet is offered a passkey ("Use Face ID, Touch ID or your device PIN") if the device supports one, or an optional password if it doesn't.
7. **MFA.** The `tp-mfa` subflow runs last.

Input from the app. The booking page passes `form = { email, displayName, applePay }` to the flow, so the customer doesn't type anything twice.

`tp-email-verify`

- Sends a 6-digit code to the customer's email and shows the address masked.
- Offers "Send a new code".
- Offers "Email me a sign-in link instead": a magic link, with a waiting screen that continues automatically once the link is clicked.

`tp-profile`

- If the account already has a name, the subflow does nothing.
- If the booking form passed a name in, it is saved silently.
- Otherwise the customer sees one question: "Nearly there — what should we call you?"

`tp-mfa`

- Customers who have an authenticator app enrolled enter its code.
- New customers are offered enrollment with a QR code and a manual setup code. Enrollment is optional.

Branding and languages

- **Style.** The flows use a custom TravelCorp style (colors, font, rounded buttons). A second, flat variant styles the embedded User Profile widget. Both are imported as named styles, so the project's default style is unchanged.
- **French.** All four flows include French translations, managed through Descope flow localization. The app passes the language chosen in its header, which defaults to the browser language.
- **Not yet translated.** Email and SMS message templates are configured separately and are still English only.

Descope capabilities used

Capability	Where it appears
Descope Flows (embedded)	Sign-in page and booking page
Passkeys (WebAuthn)	Sign-in, post-sign-up passkey offer, My account
Social login (Apple, Google)	Welcome screens
Email OTP and Magic Link	`tp-email-verify`

Capability	Where it appears
Password (with policy)	Sign-in fallback, optional password at sign-up
Authenticator app (TOTP)	`tp-mfa` , My account
Flow A/B testing (`abTestingKey`)	Welcome-screen variants
Bot trap (honeypot)	Welcome screens
Flow localization	French for all four flows
Styles	Custom flow style and widget style
User Profile widget	My account
Family Accounts	Household bookings, dependents, acting for a dependent
Role-based access control	Customer Service and Family Admin roles
User impersonation	Customer service "view as customer"
Audit search	Customer service audit trail

Run it in your own Descope project

Prerequisites

- A Descope project and a management key (console → Company → Management Keys)
- Node 24 and pnpm

1. Configure the Descope console

1. **Flows → Import flow:** import the subflows first (`tp-email-verify` , `tp-profile` , `tp-mfa`), then `tp-sign-up-or-in` . For French, turn on Localization for each flow (source `en` , target `fr` , manual) and import the matching file from `localization/` .
2. **Authentication Methods:**
   - Passkeys: set the domain to `localhost` (or your host).
   - Magic Link: set the redirect URL to `http://localhost:5173/login` .
   - Social Login: enable Apple and Google.
   - Password: set your password policy.
3. **Users → Families:** enable Family Accounts.
4. **Widgets:** make sure a User Profile widget exists. The app expects the ID `user-profile-widget` .

2. Run locally

```
pnpm install
cp .env.example .env    # add the project ID and management key
pnpm style:import descope/styles/travel-portal.json
pnpm style:import descope/styles/travel-portal-widget.json
pnpm seed               # demo users, roles and a sample family
pnpm dev                # web on :5173, API on :3001
```

What the seed script creates

- **Roles:** Customer Service (User Admin, Impersonate) and Family Admin (Family Impersonate Dependents). Existing roles with these names are reused, never edited.
- **Test users:** a returning guest, the guest's partner, and a customer service agent.
- **A sample family:** two guardians and three dependents (two children and a pet).

`pnpm reset` removes only what the seed created and leaves everything else in the project alone.

Notes and caveats

- **Demo data.** Bookings are held in memory and reset when the API restarts.
- **Passkeys** need the passkey domain to match the host the app is served from.
- **Apple Sign-In** needs Apple credentials configured in the Descope console. Google works with Descope's default credentials for testing.
- **Plan tiers.** Some capabilities shown here, such as Family Accounts, A/B testing and impersonation, may depend on your Descope plan. Confirm with your Descope account team.

Further reading

- Flow A/B testing: https://docs.descope.com/flows/conditions/flow-ab-testing
- Passkeys: https://docs.descope.com/auth-methods/passkeys
- Magic Link: https://docs.descope.com/auth-methods/magic-link
- User impersonation: https://docs.descope.com/user-impersonation
- Family Accounts API: https://docs.descope.com/api/management/users/impersonate-family-dependent
- User widgets: https://docs.descope.com/widgets/users
- Localization connectors: https://docs.descope.com/connectors/connector-configuration-guides/localization